ATHANASIOS GOURDOMICHALIS
ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS
Assignment in Computer Programming with C++

Assignment Prompt/ Description Specification

# INTERACTIVE APPLICATION USING A NODE NETWORK

## 1. Brief Description

The objective of the assignment is to create our own interactive application based on functionality that involves the concept of a network (graph) of nodes. The application can be either a game (existing or of our own inspiration) or a simple simulation. Our implementation is required to have graphical output and some form of user interaction using the Simple Game Graphics Library (SGG) created for the course.

## 2. Implementation

During the implementation of our application, we are called to combine knowledge gained from the lectures and carefully consider our code architecture in order to achieve a) effective code reusability, b) a unified and polymorphic way of calling methods, c) efficient utilization of ready-made STL structures, and d) high performance.

**The following goals are mandatory and will be graded:**

- **Use of the SGG library.** The integration of the SGG library and the use of the provided functions is mandatory and exclusive. Our application must not rely on any other external library for window management, **keyboard/mouse event** handling, graphics rendering, and audio playback. You may only use other libraries for additional, non-overlapping functions with SGG, such as network communication, reading/writing XML/JSON files, etc.
- **Use of dynamic memory.** In interactive applications-especially games and simulations-many entities are created and "live" for a limited time

during code execution. Such elements must be dynamically allocated in memory (using new) and destroyed when no longer needed.

- **Inheritance and polymorphism.** The various components of the application possess an internal ontological hierarchical structure. For instance, anything displayed on screen could be a VisualAsset, or anything exhibiting "node" behavior in a network/graph is a descendant of a Node class. If you have a user interface, you can apply polymorphism there as well, for example by having a generic Widget class that you specialize to create functionally specific elements like Button, Slider, etc.
- **Collections.** In an application like this, we store and manage a multiplicity of objects, either for program operations or for rendering graphics on the screen. We are required to use the most appropriate STL collections for the job to meet the needs of storage, searching, and batch method execution. **Caution:** for some of the provided collections to work properly with our custom classes, we may need to define the appropriate operators for sorting or hashing objects. It is strictly recommended not to implement custom collections for structures already provided by the STL (e.g., do not implement your own custom linked lists)
- We must have **one and only one instance of a GlobalState class** in your application, which:

a) stores data regarding the flow and state of the game/application (e.g., current level, available levels, statistics, score, raw simulation data, etc.)

b) provides the core **draw**, **init**, and **update methods** that invoke their corresponding counterparts directly or indirectly for **any active object within the interactive application**.

c) provides a centralized access point to locate elements and global application data (e.g., **`getPlayer()`**, **`getGUI()`**, **`getWindowSize()`**, **`getCanvasSize()`**, etc.).

## 3. Optional Features

Proper design and code structuring, meticulous method declarations (e.g., proper use of references, **const** arguments, and **const** methods), as well as

utilizing templated functions or classes where deemed useful, will be **evaluated positively**. It is noted that certain execution paths performing potentially heavy computational tasks may be run on separate thread(s).

*DO NOT do this for rendering or anything related to graphics and window context; these must be called from the application's main thread. Managing game objects, collision detection, enemy behavior, and other operations can be performed concurrently on other threads.*

## 4. Collisions

Inevitably, when we have entities within a game, visualized graph nodes, or interaction with interface elements, we need to check for collisions between certain elements or the superposition of a point over an area. There are multiple reasons why this functionality is necessary:

- Checking highlighting/selection of an element by the user's cursor.
- Preventing visual elements from overlapping on the canvas. For example, detecting when (and by how much) two graph nodes overlap in order to repel them or place them in distinct positions.
- Detecting collisions between game entities.

### 4.1 Point Containment Check in a Rectangle

A common check essential primarily for graphical interface elements is determining whether a point (usually the cursor) lies within a box representing the bounds of the graphical element. Assuming the graphical element has a center $(c_x, c_y)$ and dimensions $(w, h)$, a point with coordinates $(x, y)$ is inside the element when:

$$c_x - \frac{w}{2} \leq x \leq c_x + \frac{w}{2} \quad \text{AND} \quad c_y - \frac{h}{2} \leq y \leq c_y + \frac{h}{2}$$

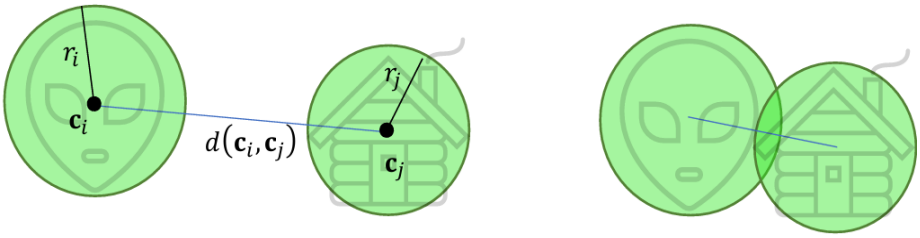
### 4.2 Point Containment Check in a Circle

A point $\boldsymbol{p} = (x, y)$ is within the "reach" of radius $\boldsymbol{r}$ of another point $\boldsymbol{c} = (\boldsymbol{c_x}, \boldsymbol{c_y})$ when:

$d(\boldsymbol{p}, \boldsymbol{c}) \leq r$, where $d(\cdot,\cdot)$ is the Euclidean distance.

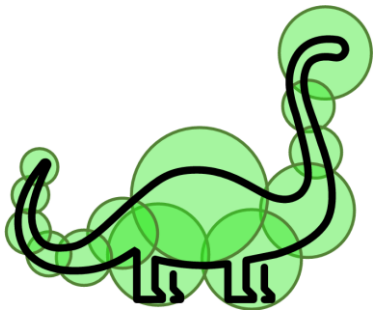
## 4.3 Disk Overlap Check

If I have an object with center $\boldsymbol{c}_i$ that can be approximated by a disk-shaped footprint with radius $r_i$ for the purposes of an overlap or collision check with other objects having a corresponding center $\boldsymbol{c}_j$ and radius $r_j$, then they overlap in the plane **if and only if**: $d(\boldsymbol{c}_i, \boldsymbol{c}_j) - r_i - r_j < 0$.




If I have fairly complex shapes, I can define multiple collision circles for them so that, by reducing their radii, I can better describe the boundaries of the object.

An object that contains multiple collision areas (circles in this case) collides with another object if at least one of its areas collides with a corresponding collision area of the other object:



## 6. Audiovisual Material

Your applications can be implemented using very basic graphics and the built-in drawing capabilities of SGG. Therefore, whether during the initial stages of your implementation (where you primarily test functionality) or if you lack the capacity or time to focus on content "presentation", you may use the ready-made drawing primitives provided by SGG to display simple forms and shapes on the screen.

If, however, you wish to include your own assets or ready-made bitmaps downloaded from the internet, the PNG format is more than sufficient for loading and drawing images on top of the basic primitives, as it supports an alpha (transparency) channel and is compatible with all popular and free image editing and conversion software. Ensure your files are sized appropriately to fit the application's requirements. For example, loading an image found online with a resolution of 4400x3300 pixels as a background is excessively large, unnecessarily wasting a) memory, b) loading time (especially in debug mode), and c) disk space as well as space in the final zip archive. Using an image editing program, resize it to dimensions suitable for your window. E.g., for a 1024x768 window, resizing the image to 1067x800 in our example is well-suited, fully covering the expected window resolution while preserving the original aspect ratio. Note: sizing your images to power-of-two dimensions (e.g., 1024x512, 64x64) allows the library to load them faster, as the loading function would otherwise need to convert them to the nearest powers of 2.

## 7. Assignment Submission

Our assignment must be uploaded as a zip file that must include:

- **The assignment code. Note:** from the SGG library, include only the 2 header files required to compile your program (**graphics.h, scancodes.h**), not the entire library codebase!
- **The assets used by your application** in the appropriate folders so that the built executable can locate them at runtime. **Attention**: if you use many and/or large image or sound files and your zip size increases significantly (**>4MB**), prefer to upload the assets folder separately to an external link (that does not expire—do not use e.g. WeTransfer) and include a txt file in your zip describing a) the URL of the external link, b) the relative directory path from the executable where the application expects to locate these data files. For example, if the executable is located in folder D:\game\bin\ and looks for data files in the directory D:\game\bin\assets, specify the relative path .\assets in the txt file.
- **The necessary files for building the assignment code**, e.g., the solution (.sln) and Project file (.vcxproj) in the appropriate subfolders if needed, or a makefile or build script.

**Do not upload:**

- Dynamic link library files (DLLs, SOs).
- The .lib files of SGG.
- Temporary files generated during the build process of the application, which may (especially in the case of Visual Studio) be quite large. Such

files include *.pdb, *.tmp, *.obj, *.ilk. **Attention**: some of these are hidden, such as the .vs directory.

## 8. Assignment Grading

The grading of the assignments, out of a maximum score of 4, is based on the following criteria:

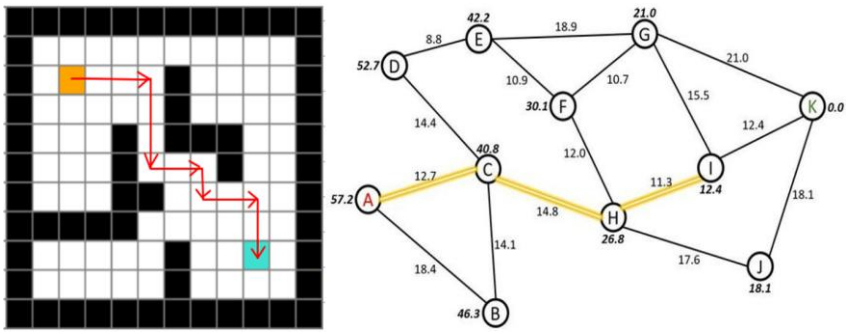
| Criterion | Impact | Comment |
|---|---|---|
| **Non-use of the SGG library** | Assignment rejection | The **exclusive** use of the SGG library is mandatory. |
| **Submission of an assignment that does not fall thematically within the specifications** | Assignment rejection | If you are unsure whether your chosen topic is an "arcade game" or a related game, ask. |
| **Non-use of inheritance** | Up to -1.0 | See section "Implementation". |
| **Non-polymorphic invocation of common methods** | Up to -1.0 | See section "Implementation". |
| **Failure to declare and implement a base class for elements with common functionality and a** GlobalState **state class** | Up to -0.5 | See section "Implementation". |
| **Unsatisfactory application design** | Up to -0.5 | See section "Implementation". |
| **Non-functional user interface / game flow control** | Up to -0.5 | - |

| | | |
|---|---|---|
| **Implementation of desirable (optional) features** | Up to +0.5 | See section "Implementation". |

It is noted that, based on the above, a well-executed assignment that has implemented features beyond the mandatory ones may exceed a grade of 4.

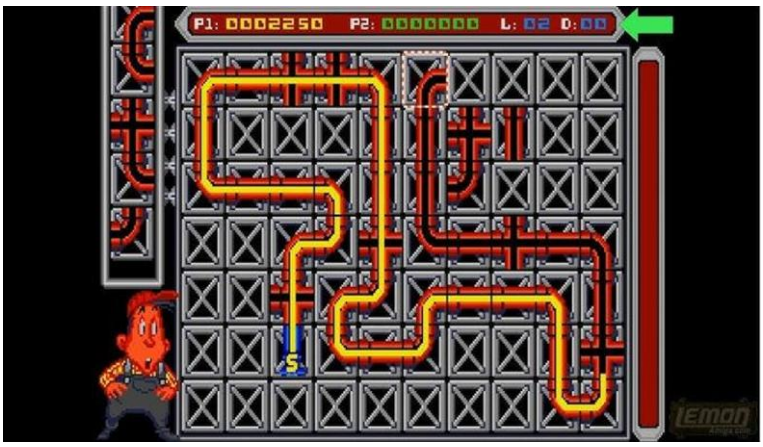
# 9. Potential Ideas

- Routing on a map or grid with obstacles – A* algorithm (Difficulty: Medium)

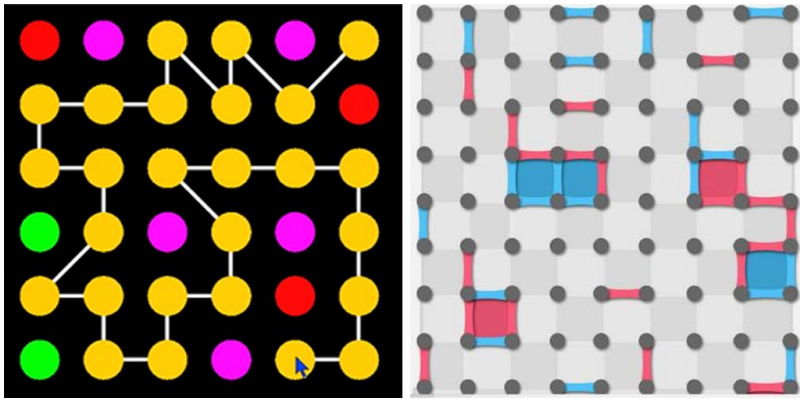



- Water network and pipe load simulation (Difficulty: Medium)

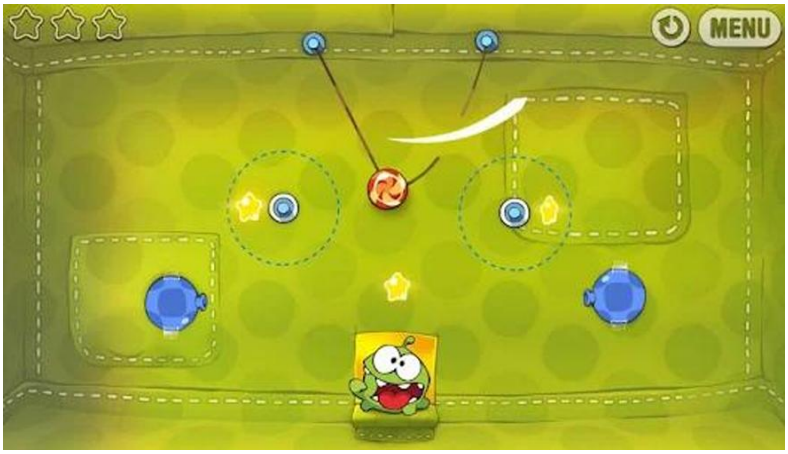
- A "Pipe-mania" style game (Difficulty: Medium)



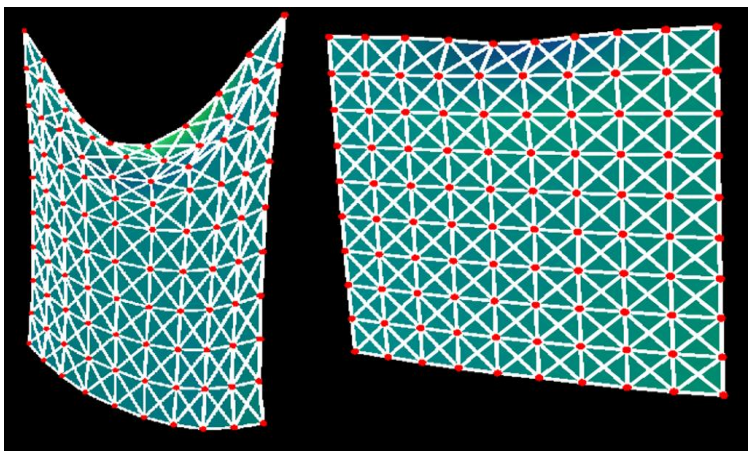
- A "connect the dots" or "dots and boxes" style game (Difficulty: Medium)



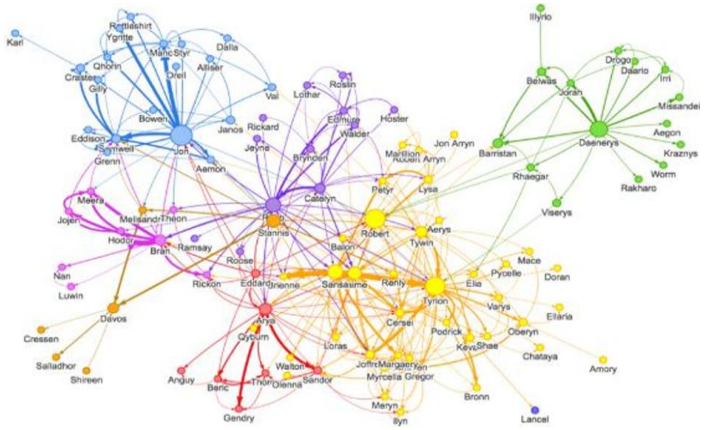
- A "Cut the Rope" style game (Difficulty: High)



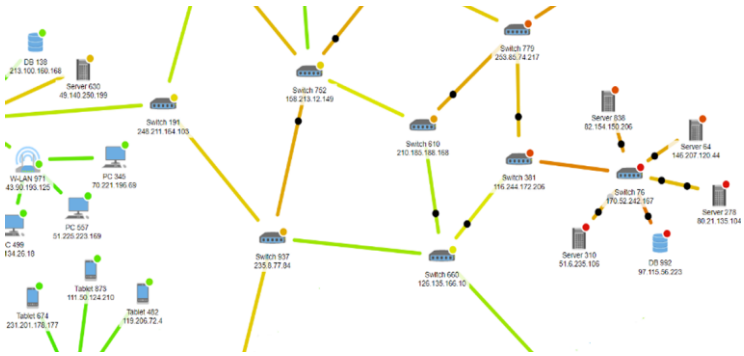

- Interactive 2D cloth simulation, optionally with tearing support (Difficulty: High)



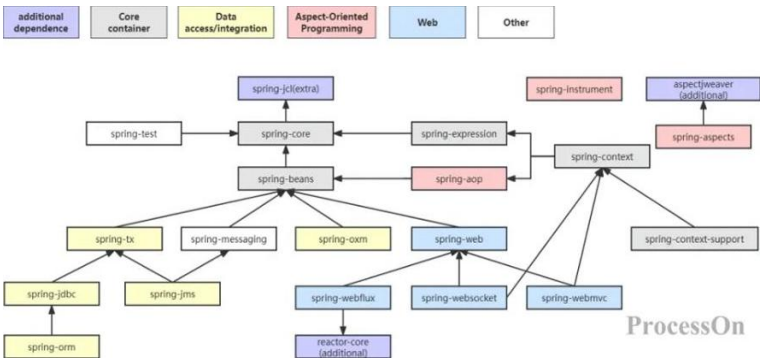
- Social network visualization (Difficulty: Medium)



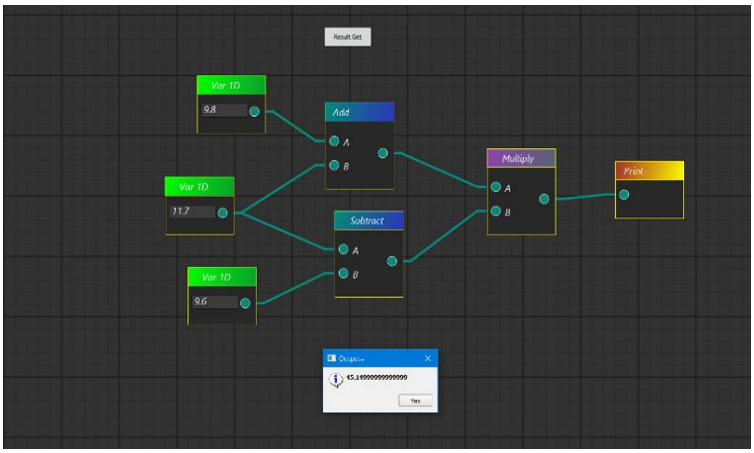

- Network traffic routing simulation (Difficulty: High)



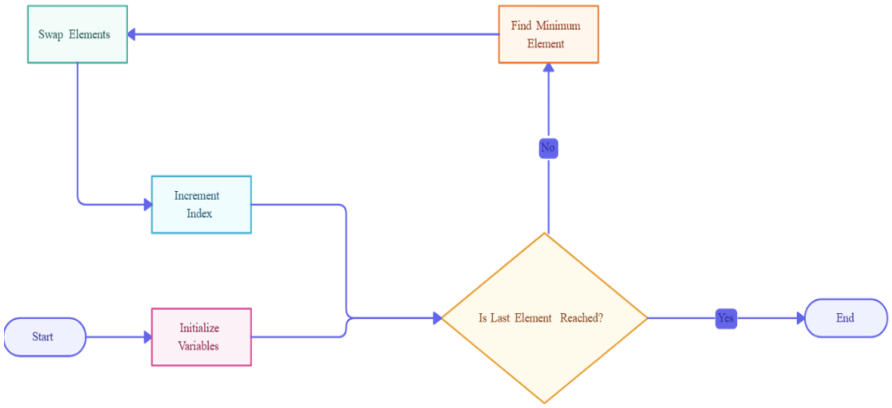

- Dependency graph visualization (Difficulty: Medium)



- Educational visual coding application




- Program flow visualization / visual debugging application




Creativity is rewarded. We feel free to come up with our own applications!